IT 112: Web Systems & Technologies

BSIT-2B 2nd Sem

S.Y. 2025-2026

Final Project Proposal
Phase 4 Documentation for the Quick Reserve :Online Bus & Travel Booking System A Final Project by the <Tech_ Vanguard />(Week 13)

Submitted By:

Nathaniel O. Raynada

Anna Mae P. Broma

Camille T. Bercilla

Mariel O. Odoño

John Edmar L. Bongcodin

Justin James Dasallas

Submitted To:

Dr. Guillermo V. Red, Jr, DIT

April 2026

I. Overview of Form and API Integration

The QuickReserve system includes form and API integration to allow the frontend and backend to work together as one complete system. This integration enables the application to accept user input from the interface and connect it directly to the server and database.

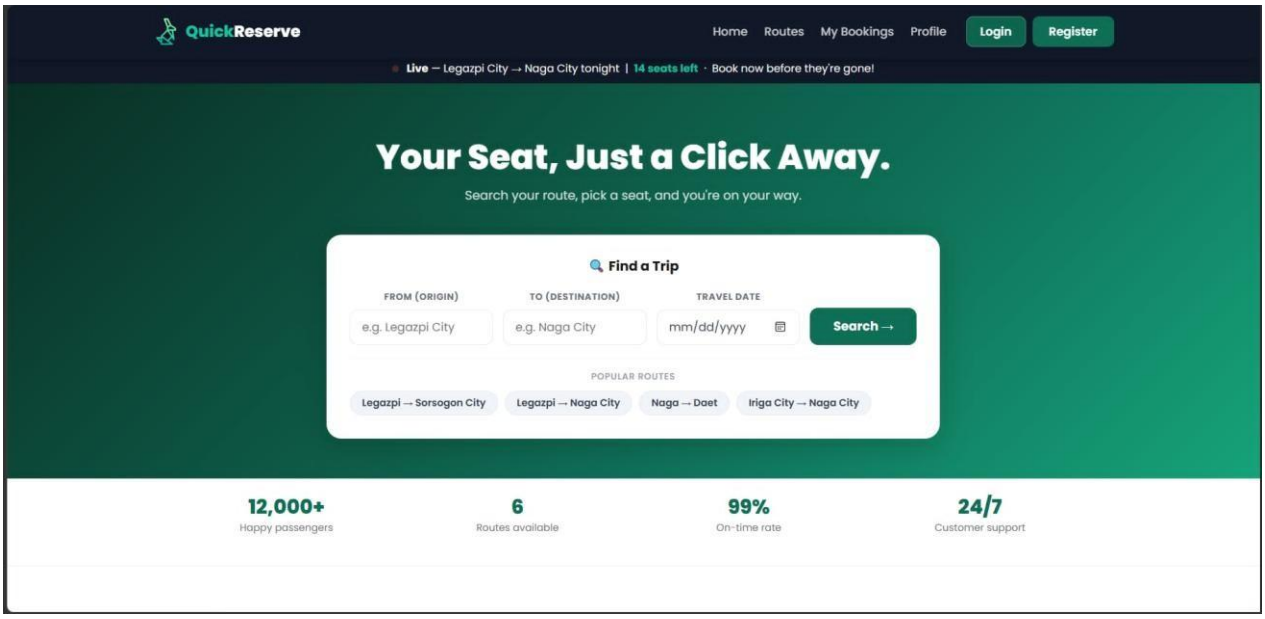
Through this setup, the system supports important features such as user registration, login, and booking. The frontend is responsible for collecting and sending user data, while the backend handles the processing and storage of this information in the **MongoDB Atlas** database. This connection ensures that all data entered by users is properly saved and managed.

The system uses API endpoints to allow communication between the frontend and backend. These APIs serve as the bridge that transfers data and ensures that requests and responses are handled correctly. With this integration, the system becomes more dynamic and interactive, allowing users to perform actions and receive immediate feedback.

In the end, form and API integration plays a key role in making QuickReserve functional and reliable. It allows smooth communication between different parts of the system and ensures that user data is handled efficiently, supporting a better and more convenient booking experience.

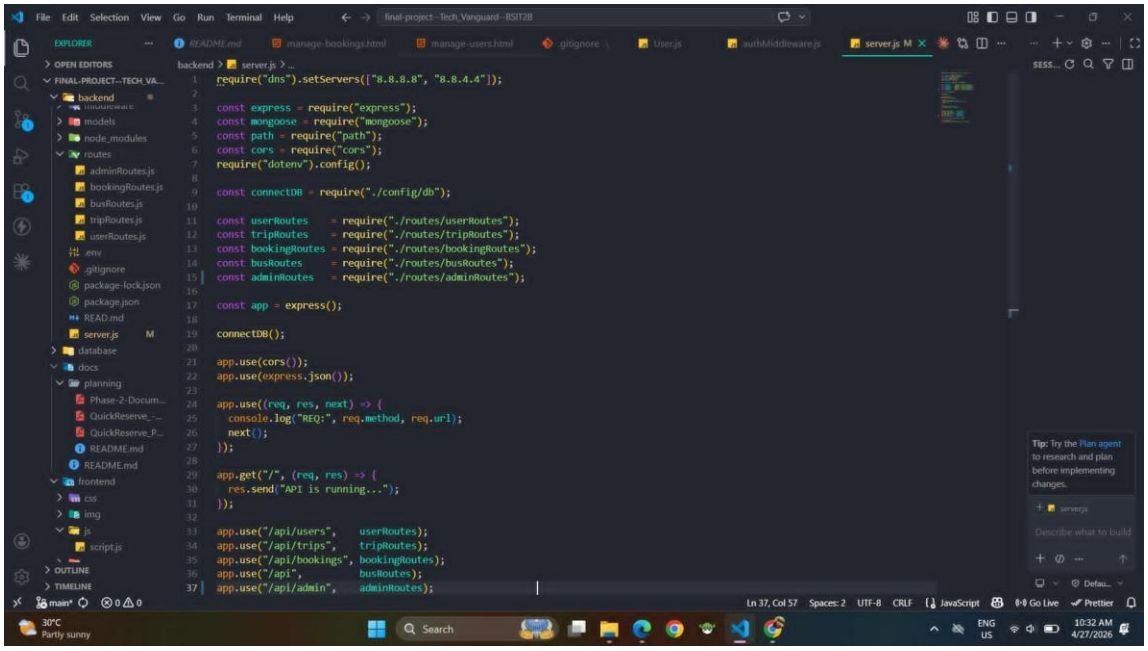
II. HTML Form Interface

The HTML form interface of the QuickReserve system. It contains input fields where users provide necessary details such as name, email, and password. The form serves as the primary method for gathering user information before it is sent to the backend.



III. JS Code used for API connection

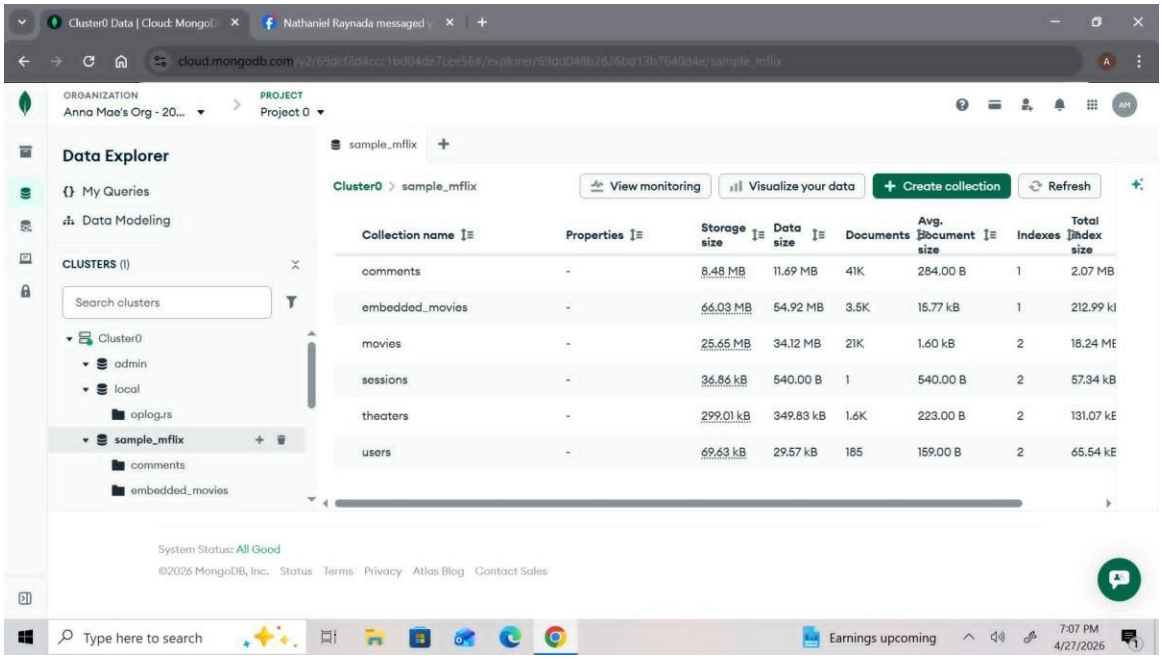
This JavaScript code used for the backend server of the system. It utilizes Node.js and Express to define API routes such as users, trips, bookings, buses, and admin. The code also handles incoming requests, connects to the database, and runs the server on a specified port to enable communication between the frontend and backend.



```
require("dns").setServers(["8.8.8.8", "8.8.4.4"]);

const express = require("express");
const mongoose = require("mongoose");
const path = require("path");
const cors = require("cors");
require("dotenv").config();

const connectDB = require("./config/db");

const userRoutes    = require("./routes/userRoutes");
const tripRoutes    = require("./routes/tripRoutes");
const bookingRoutes = require("./routes/bookingRoutes");
const busRoutes     = require("./routes/busRoutes");
const adminRoutes   = require("./routes/adminRoutes");

const app = express();

connectDB();

app.use(cors());
app.use(express.json());

app.use((req, res, next) => {
  console.log("REQ:", req.method, req.url);
  next();
});

app.get("/", (req, res) => {
  res.send("API is running...");
});

app.use("/api/users",    userRoutes);
app.use("/api/trips",    tripRoutes);
app.use("/api/bookings", bookingRoutes);
app.use("/api",          busRoutes);
app.use("/api/admin",    adminRoutes);
```

IV. MongoDB Document Inserted

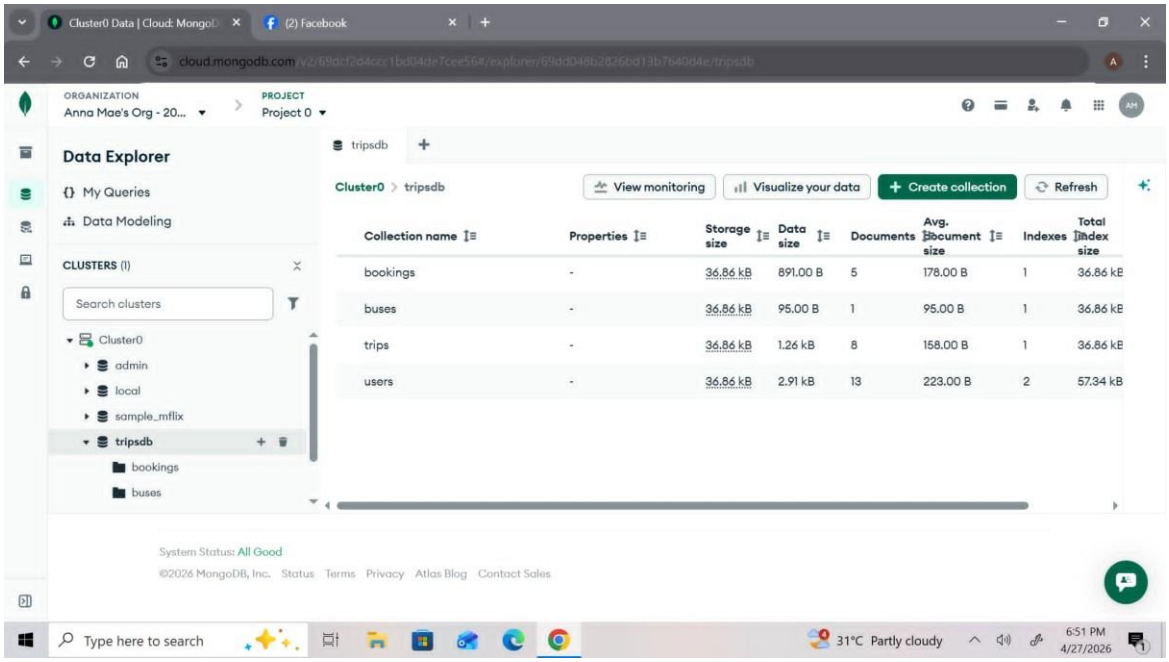
The data stored in MongoDB Atlas after form submission. It confirms that the user input is successfully saved in the database, proving that the backend connection is working.



Cluster0 > sample_mflix

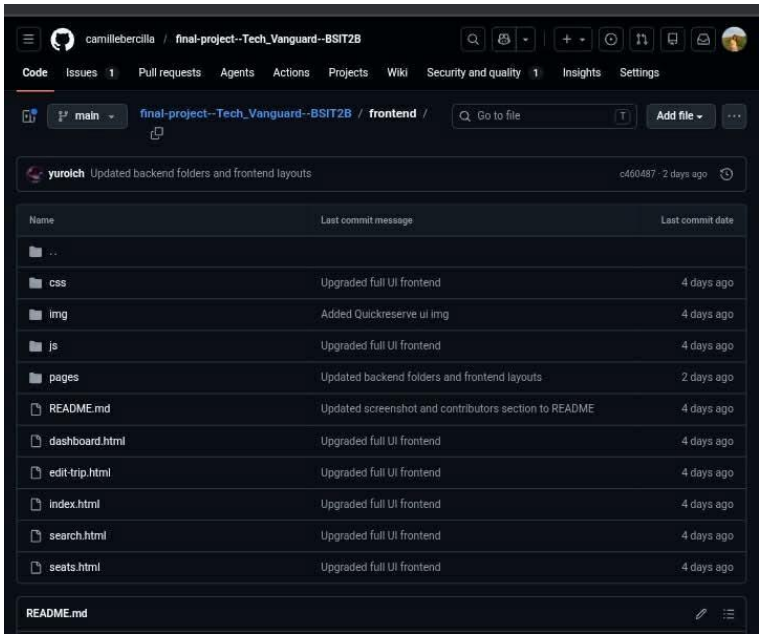
View monitoring | Visualize your data | + Create collection | Refresh

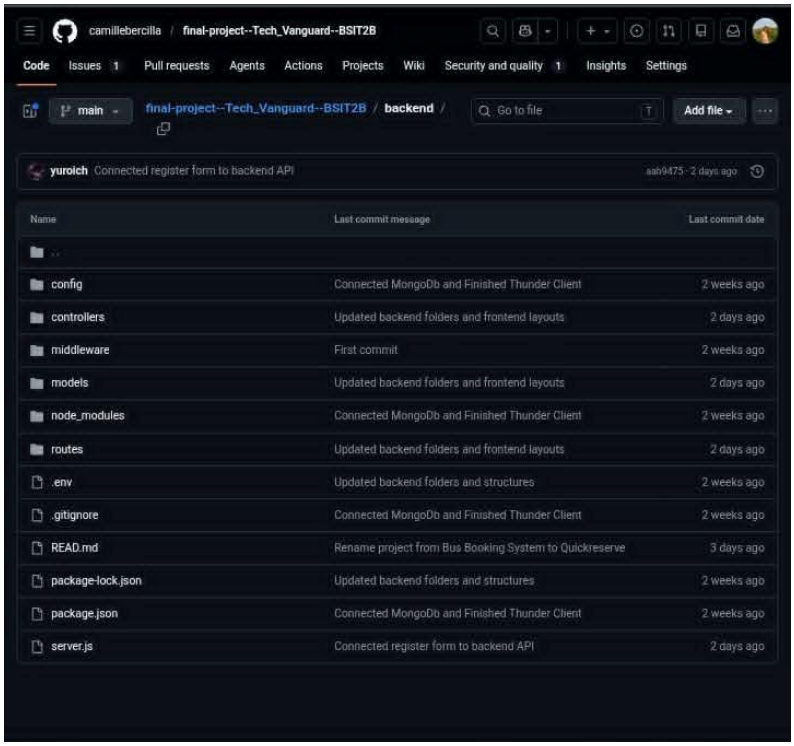
Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total Index size
comments	-	8.48 MB	11.69 MB	41K	284.00 B	1	2.07 MB
embedded_movies	-	66.03 MB	54.92 MB	3.5K	15.77 kB	1	212.99 kI
movies	-	25.65 MB	34.12 MB	21K	1.60 kB	2	18.24 ME
sessions	-	36.86 kB	540.00 B	1	540.00 B	2	57.34 kB
theaters	-	299.01 kB	349.83 kB	1.6K	223.00 B	2	131.07 kE
users	-	69.63 kB	29.57 kB	185	159.00 B	2	65.54 kE



V. GitHub Commits Fronted & Backend

Project's commit history on GitHub. It reflects updates made to both frontend and backend components, demonstrating collaboration and proper version control practices.





VI. Short reflections from each group member (3–5 sentences)

Each member was assigned a role to help in the development, testing, and documentation of the Quick Reserve: Online Bus & Travel Booking System.

Nathaniel O. Raynada – Project Manager

As the frontend developer and project leader, I was responsible for connecting the HTML forms to the backend API while also guiding the team and ensuring tasks were well-coordinated. I implemented form handling using JavaScript and fetch(), which helped me understand how frontend and backend systems communicate in real-time. I also contributed to the backend by helping test and refine API routes, strengthening my overall understanding of full-stack development.

Anna Mae P. Broma – Back-End Developer

As the backend developer in Phase 4, I was responsible for ensuring that the API endpoints were working properly and that form data from the frontend was successfully inserted into the MongoDB database. I learned how to handle POST requests, validate incoming data, and return appropriate JSON responses for both success and error cases. This phase helped me understand the importance of seamless integration between frontend and backend in building a functional web application.

Camille Bercilla – GitHub Manager

I helped verify whether the uploaded or pushed folders were correct before committing. I ensured that all new and updated files including HTML pages, JavaScript files for handling forms, and

backend route controllers were properly included. And we pushed everything to GitHub with clear and organized commit messages.

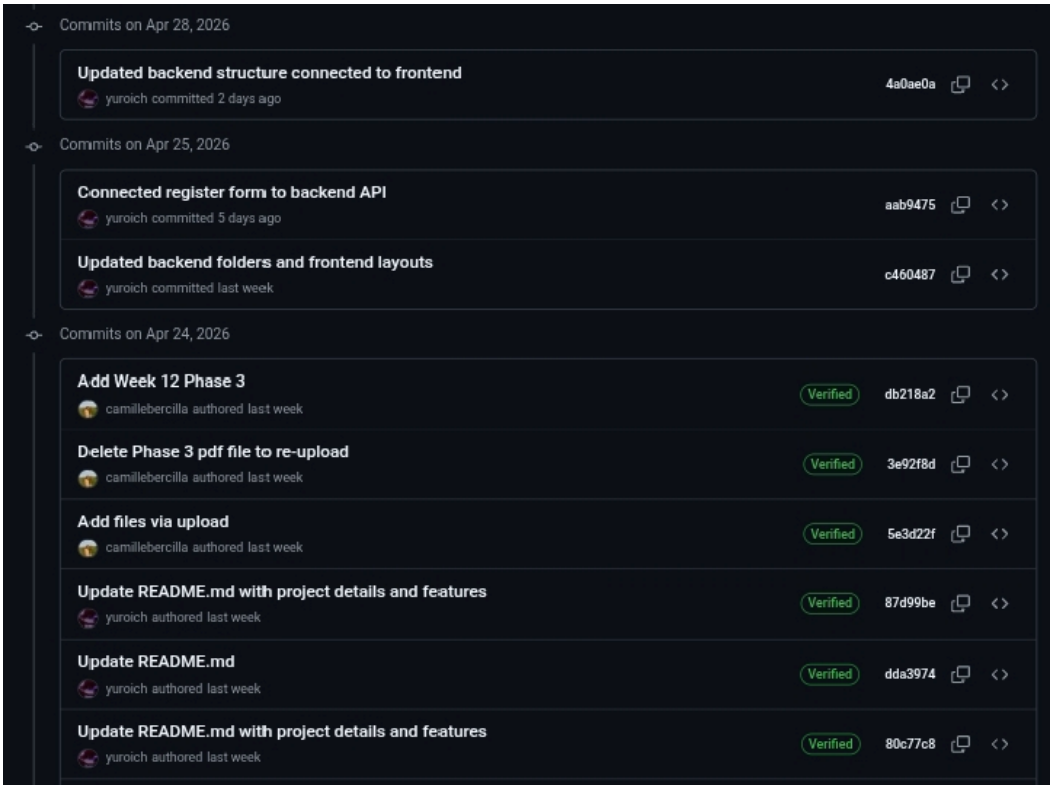
Mariel O. Odoño – Documentation Officer

I was assigned to handle the project documentation by gathering necessary screenshots and providing clear explanations of how the system's features fulfill the required functions. I also arranged and summarized the technical details into a well-organized report. Through this role, I recognized how essential documentation is in presenting and supporting the overall system development.

John Edmar Bongcodin – System Tester

I worked as a tester and debugger by checking if the forms were properly connected to the backend and if data was submitted correctly. I tested validations and error messages to ensure the system worked smoothly. This phase helped me improve my debugging and problem-solving skills.

Justin Dasallas – Database Manager

I supported the documentation process by gathering screenshots of the system and adding relevant code examples. I made sure that all written details were consistent with the system's actual behavior. I also helped evaluate the user interface to ensure it is simple, clear, and easy to navigate.

VII.GitHub Commits